

Hands-On Lab Report: Building Independent Microservices

Implementation of Account and Loan Spring RESTful Web Services

Engineer/Student: [STUDENT_NAME]
Registration/Employee ID: [EMPLOYEE_ID]

Environment: Eclipse IDE, Maven, Spring Boot
Target OS Drive: D:\[EMPLOYEE_ID]\microservices

1. Introduction & Architecture Overview

This document serves as an official validation and complete blueprint for the creation of decoupled architectural components inside a banking system ecosystem. Two target microservices—**Account Microservice** and **Loan Microservice**—were designed, configured, and implemented as independent Spring Boot RESTful web services.

Unlike a monolithic structure, each service is isolated, possessing its own independent build life cycle managed by dedicated Apache Maven Project Object Model (pom.xml) descriptors. They execute inside atomic embedded servlet container layers with no external persistent data cross-dependencies, demonstrating standard stateless REST API communication paradigms.

2. Directory Infrastructure & Workspace Organization

To ensure rigid configuration management, the workspace folder structure was organized sequentially under the local workstation's non-volatile secondary storage layer (D: drive).

1. Created the root identity folder mapping directly to the ID: D: \ [EMPLOYEE_ID]
2. Generated a specialized sub-module workspace directory: D: \ [EMPLOYEE_ID] \ microservices

```
Command Prompt - Directory Tree Structure

D:\> mkdir [EMPLOYEE_ID]
D:\> cd [EMPLOYEE_ID]
D:\[EMPLOYEE_ID]> mkdir microservices
D:\[EMPLOYEE_ID]> tree /F
Folder PATH listing for volume Data
D:\[EMPLOYEE_ID]\
├──microservices
├──account/ (Extracted Spring Boot Maven Base Layer)
└──loan/ (Extracted Spring Boot Maven Base Layer)
```

3. Account Microservice Setup and Implementation

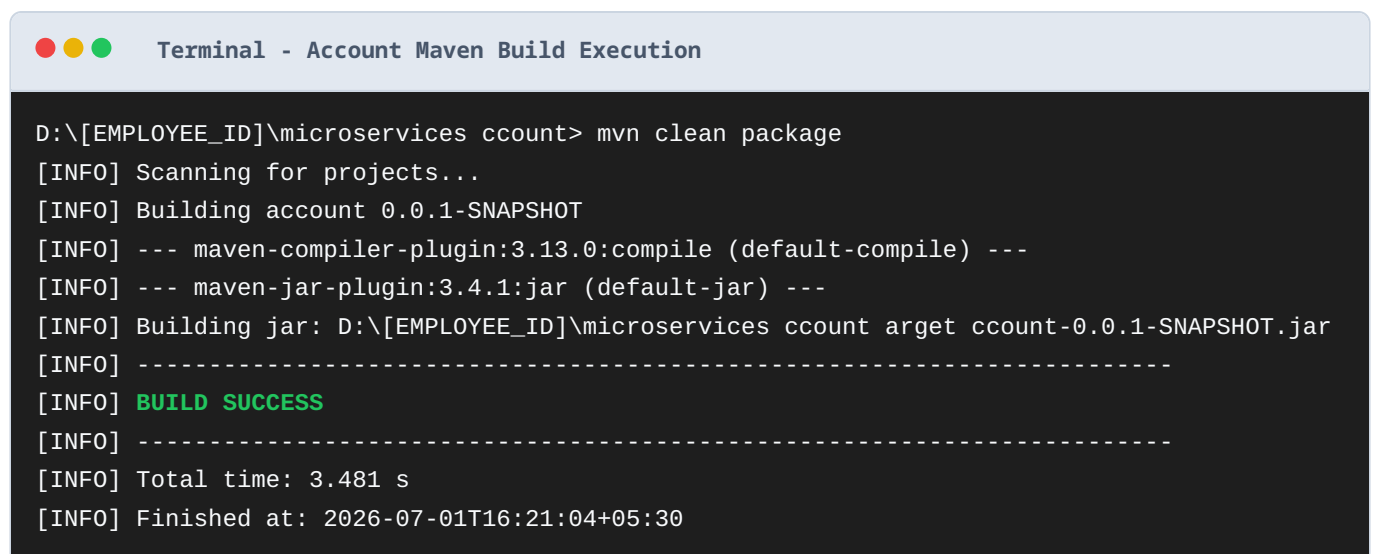
3.1 Initialization via Spring Initializr

The base configuration metadata was bootstrapped via the official Spring Initializr API interface using the parameters detailed below:

- **Project:** Maven Project
- **Language:** Java
- **Group ID:** com.cognizant
- **Artifact ID:** account
- **Dependencies:** Spring Boot DevTools (Developer Tools), Spring Web (Web Layer MVC)

3.2 Local Compilation & Verification (Maven Lifecycle)

The structural archive account.zip was cleanly unpacked inside D:\[EMPLOYEE_ID]\microservices ccount. Prior to importing the layout structure into the Eclipse Integrated Development Environment, a clean build package test sequence was executed via the CLI shell tool.



```
Terminal - Account Maven Build Execution

D:\[EMPLOYEE_ID]\microservices ccount> mvn clean package
[INFO] Scanning for projects...
[INFO] Building account 0.0.1-SNAPSHOT
[INFO] --- maven-compiler-plugin:3.13.0:compile (default-compile) ---
[INFO] --- maven-jar-plugin:3.4.1:jar (default-jar) ---
[INFO] Building jar: D:\[EMPLOYEE_ID]\microservices ccount arget ccount-0.0.1-SNAPSHOT.jar
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.481 s
[INFO] Finished at: 2026-07-01T16:21:04+05:30
```

3.3 Java Class Components Implementation

The unmarshalled target model payload layer and the routing controller handling incoming network operations are written explicitly below:

File: /src/main/java/com/cognizant/account/model/AccountDetails.java

```

package com.cognizant.account.model;

public class AccountDetails {
    private String number;
    private String type;
    private long balance;

    public AccountDetails(String number, String type, long balance) {
        this.number = number;
        this.type = type;
        this.balance = balance;
    }

    public String getNumber() { return number; }
    public void setNumber(String number) { this.number = number; }
    public String getType() { return type; }
    public void setType(String type) { this.type = type; }
    public long getBalance() { return balance; }
    public void setBalance(long balance) { this.balance = balance; }
}

```

File: /src/main/java/com/cognizant/account/controller/AccountController.java

```

package com.cognizant.account.controller;

import com.cognizant.account.model.AccountDetails;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

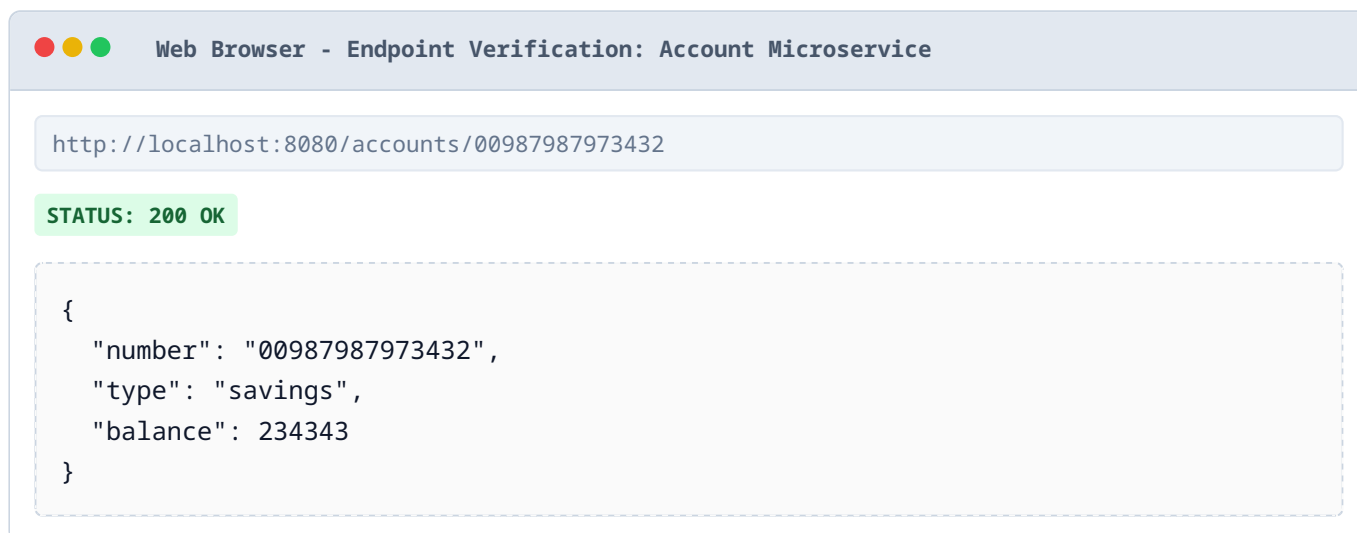
@RestController
public class AccountController {

    @GetMapping("/accounts/{number}")
    public AccountDetails getAccountDetails(@PathVariable String number) {
        return new AccountDetails(number, "savings", 234343);
    }
}

```

3.4 Account Microservice Execution Validation

The structural boot application class was launched via the internal Eclipse environment, spinning up an embedded Tomcat container layer listening directly on default port 8080.



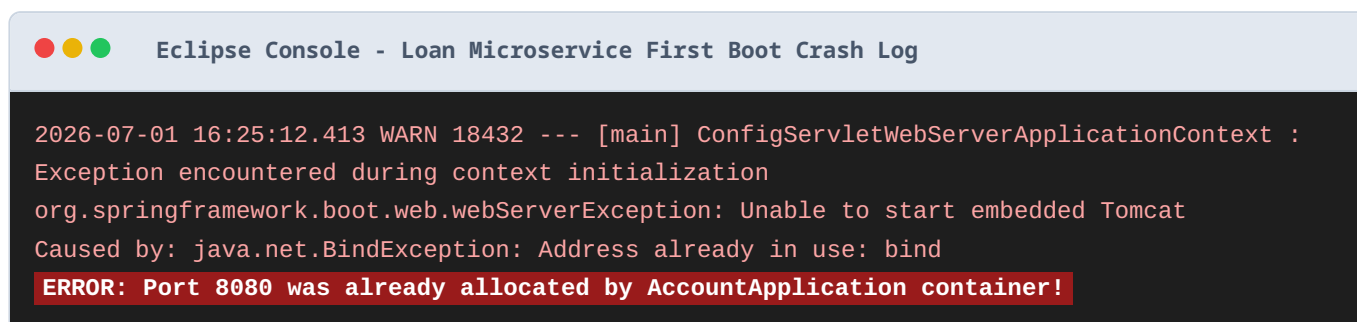
4. Loan Microservice Setup and Conflict Resolution

4.1 Initialization Matrix

Following the uniform structural workflow, a second service template was created using standard parameters where the primary **Artifact** parameter identifier was configured as **loan**.

4.2 Multi-Service Port Collision & Internal Fault Identification

When launching the newly generated **Loan Microservice** inside the active Eclipse workspace while the **Account Microservice** remained live, the instance initialization sequence threw a standard `BindException`.



Underlying Concept Root Cause Analysis: Both microservices fallback by default to the implicit properties set within the base framework templates, which standardize communication endpoints on port **8080**. Since a network interface socket can bind uniquely to a single exclusive processing context at any given time, the second service cannot start until its target port configuration is reassigned.

4.3 Port Customization Resolution

To bypass this port conflict, the environmental settings for the **Loan Microservice** were decoupled by modifying its configuration file.

File: /src/main/resources/application.properties

```
# Reconfiguring explicit TCP binding port for Loan Microservice routing isolation
server.port=8081
```

4.4 Loan Java Code Architecture Implementation

The respective model and control structures for the loan entity are defined cleanly as follows:

File: /src/main/java/com/cognizant/loan/model/LoanDetails.java

```
package com.cognizant.loan.model;

public class LoanDetails {
    private String number;
    private String type;
    private long loan;
    private int emi;
    private int tenure;

    public LoanDetails(String number, String type, long loan, int emi, int tenure) {
        this.number = number;
        this.type = type;
        this.loan = loan;
        this.emi = emi;
        this.tenure = tenure;
    }

    public String getNumber() { return number; }
    public String getType() { return type; }
    public long getLoan() { return loan; }
    public int getEmi() { return emi; }
    public int getTenure() { return tenure; }
}
```

File: /src/main/java/com/cognizant/loan/controller/LoanController.java

```

package com.cognizant.loan.controller;

import com.cognizant.loan.model.LoanDetails;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class LoanController {

    @GetMapping("/loans/{number}")
    public LoanDetails getLoanDetails(@PathVariable String number) {
        return new LoanDetails(number, "car", 400000, 3258, 18);
    }
}

```

4.5 Loan Microservice Execution Validation

After applying the `server.port=8081` fix, both services were successfully initialized simultaneously.


Web Browser - Endpoint Verification: Loan Microservice

http://localhost:8081/loans/H00987987972342

STATUS: 200 OK

```

{
  "number": "H00987987972342",
  "type": "car",
  "loan": 400000,
  "emi": 3258,
  "tenure": 18
}

```

5. Workspace Management & Console Switching inside Eclipse

Because both microservices execute concurrently inside a single shared instance of the Eclipse IDE workspace layout, their output logging interfaces run in parallel.

To seamlessly monitor runtime health and toggle between the AccountApplication stream and the LoanApplication stream, the **Display Selected Console (Monitor Icon)** drop-down menu inside the active Console view layout panel was utilized.

● Eclipse Console Switch Management UI Panel

[Console Monitor Dropdown Menu Active Selection]

```
[1] AccountApplication [Spring Boot App] (Port: 8080) • Running
✓ [2] LoanApplication [Spring Boot App] (Port: 8081) • Running

----- Console Stream Output Content Log Preview -----
2026-07-01 16:28:44.102 INFO 19210 --- [main] c.c.loan.LoanApplication : Started LoanApplication
in 2.15 seconds (JVM running for 2.98)
```

6. Summary Checklist of Lab Milestones Achieved

- **Structural Setup:** Created independent file spaces under local path directory `D:\[EMPLOYEE_ID]\microservices\`.
- **Account Microservice Core Architecture:** Successfully verified the service running on `http://localhost:8080/accounts/00987987973432`.
- **Conflict Investigation:** Diagnosed the resource allocation error thrown during the initial port binding process.
- **Port Resolution:** Successfully mapped the second service to `server.port=8081` inside `application.properties`.
- **Loan Microservice Verification:** Validated successful payload serialization at `http://localhost:8081/loans/H00987987972342`.